[AWS] 맥도날드로 배우는 Serverless

지난번엔 VPC와 3-Tier 아키텍처를 공부하면서 EC2, ECS, EKS, Lambda 등 여러 컴퓨팅 서비스를 비교해 봤다.

그때 나는 이해가 안 되는 개념들을 전부 맥도날드에 비유해서 생각했는데 이번에도 어김없이 맥도날드가 등장할 예정이다. 어쩌다 보니 AWS 공부하는 건지 맥도날드 창업을 준비하는 건지 모르겠네ㅋㅋ

아무튼 이번에 공부할 주제는 **Serverless** 다.

지난번에는 Lambda를 단순히 *주문이 들어오면 그때 직원을 호출하는 방식* 정도로 이해했었다.

그런데 서버리스 아키텍처를 조금 더 공부하다 보니 Lambda 하나만 안다고 끝나는 이야기가 아니었다.

Lambda는 어떻게 요청을 전달받는지, 요청이 동시에 몰리면 어떻게 되는지, 처리 중 오류가 발생하면 누가 복구하는지 등 생각보다 고민할 게 많았다.

그래서 이번에는 맥도날드 주문 시스템을 하나 만들어 본다고 가정하면서 **Serverless → 동기·비동기 처리 → 이벤트 기반 아키텍처 → 데이터 정합성과 장애 복구**까지 연결해 보려고 한다.

1. Serverless 말 그대로 이해해도 될까?

서버리스라고 하면 정말 서버가 없는 걸까요?

하고 친구가 전에 서버리스를 주제로 발표를 한 적이 있다. 보통 이런 뻔한 질문에 대한 답들은 예상과 다르다.

Serverless는 개발자가 서버를 직접 프로비저닝하고 관리하는 부담을 줄이고 클라우드 제공업체가 실행 인프라의 운영을 담당하는 클라우드 컴퓨팅 방식이다.

여기서 프로비저닝이란 서버나 네트워크처럼 서비스 운영에 필요한 자원을 준비하고 할당하는 과정을 말한다. 쉽게 말해 서버는 여전히 존재하고 그냥 개발자가 그 서버를 직접 관리할 필요가 크게 줄어드는 것이다.

다시 맥도날드로 ㅎㅎ

내가 맥도날드 점주라고 생각해 보자. 아 벌써 행복하다.

기존 EC2 방식에서는 매장과 주방을 준비하고 직원을 고용해서 배치해야 한다. 손님이 늘어나면 직원을 더 투입하고 손님이 줄어들면 인원을 줄여야 한다.

AWS에서는 이 역할을 EC2와 ASG를 이용해 구현할 수 있다. 그런데 주방 시설 관리와 인력 배치를 전문 업체에서 알아서 담당해 준다면?

나는 어떤 주문을 어떻게 처리해야 하는지에 더 집중할 수 있을 것이다. 이것이 Serverless를 이해하기 위한 첫 번째 비유다.

EC2와 Lambda의 차이

구분	EC2 + ASG	AWS Lambda
실행 단위	가상 서버 인스턴스	함수 실행 환경
서버 관리	사용자가 OS 등 관리	AWS가 실행 인프라 관리
확장 방식	인스턴스 수 조절	동시 실행 환경 자동 확장
비용 구조	주로 인스턴스 실행 시간	주로 호출 수와 실행 자원
적합한 상황	지속 실행, 세밀한 환경 제어	이벤트 기반, 간헐적·변동성 있는 작업

여기서 하나 궁금했던 점은

> EC2도 ASG로 자동 확장할 수 있는데 굳이 Lambda를 사용할 이유가 있을까?

둘 다 자동 확장이 가능하지만 관리하는 대상과 **책임 범위**가 다르다.

ASG는 EC2 인스턴스의 개수를 조절하는 서비스다. 반면 Lambda는 개발자가 서버 인스턴스를 운영하지 않고 코드를 실행하도록 해준다.

자동 확장이라는 공통점은 있는데 서버를 관리하는 방식 자체가 다른 것이라 굳이굳이 비교를 막 할 수는 없는 것이다.

그럼 Lambda가 무조건 효율적일까?

이번엔 24시간 내내 손님이 끊이지 않는 맥도날드를 생각해 보자. 이거 저번에 EC2 이야기 할 때 장사 잘되는 맥날 vs 장사 잘 안되는 구석진 곳에 위치한 맥날 체인점으로 이해했었던 것 같다.

여튼 24시간 내내 돌리는 맥도날드 체인점에선 주방 직원이 계속 주문을 처리해야 한다면 필요할 때마다 작업 환경을 사용하는 것보다 일정한 조리 설비를 계속 운영하는 편이 더 효율적일 수도 있다.

실제 AWS에서도 비슷하다.

Lambda는 사용하지 않을 때 일반적인 함수 실행 비용이 발생하지 않는다는 장점이 있지만 지속적으로 높은 부하가 발생한다면 EC2가 비용이나 성능 측면에서 유리할 수 있다.

게다가 Lambda에는 실행 시간 제한도 있고 새로운 실행 환경을 준비하면서 지연이 생길 수도 있다.

따라서 **Serverless가 모든 환경에서 더 좋은 선택지는 아니다.**

중요한 것은 **워크로드**(=애플리케이션이 수행하는 작업의 양과 특성)에 따라 적절한 컴퓨팅 방식을 선택하는 것이다.

2. Lambda는 주문이 들어왔다는 사실을 어떻게 알까?

지난번 Lambda를 공부하면서 이렇게 이해했다.

> "손님이 없으면 직원도 없다가 주문이 들어오면 직원을 부르는 방식"

GPT쌤과 1:1 과외를 하다보니 솔직히 말해서 말들이 따로 노는 것도 있고 조금 어색하긴 하다.

그래서 Lambda가 계속 켜져 있는 것도 아닌데 주문이 들어왔다는 사실을 어떻게 알고 실행되는 걸까라고 묻는다면 Lambda는 다른 서비스가 전달하는 이벤트나 호출을 통해 실행될 수 있다고 대답하면 된다.

AWS Lambda

AWS Lambda는 서버를 직접 프로비저닝하거나 관리하지 않고 이벤트 또는 호출에 따라 코드를 실행하는 서버리스 컴퓨팅 서비스다.

여기서 이벤트는 시스템에서 발생한 사건이나 작업을 유발하는 입력을 의미한다. 예를 들어 사용자의 HTTP 요청, 파일 업로드, 데이터 변경 등이 Lambda 실행의 계기가 될 수 있다.

이번 맥도날드에서는 키오스크에서 주문 버튼을 누르는 상황을 생각해보자.

```
손님: 빅맥 하나 주세요.
          |
          ▼
     API Gateway
          |
          ▼
       Lambda
          |
          ▼
      DynamoDB
```

여기서 세 가지 AWS 서비스가 등장한다.

API Gateway — 주문 받는 키오스크

Amazon API Gateway는 클라이언트가 API에 접근할 수 있는 엔드포인트를 제공하고 요청을 연결된 백엔드로 전달하는 관리형 서비스다.

맥도날드에서는 키오스크 주문 창구에 비유할 수 있다. 손님이 주문 버튼을 누르면 API Gateway가 요청을 받아 Lambda에 전달하는 것이다.

Lambda — 실제 주문 처리하는 직원

Lambda는 전달된 주문 정보를 검증하고 필요한 작업을 수행한다. 예를 들어 빅맥 주문이 올바른지 확인하고 주문 정보를 DB에 저장하는 코드를 실행할 수 있다.

DynamoDB — 주문 관리 장부

Amazon DynamoDB는 AWS가 제공하는 완전 관리형 서버리스 NoSQL 데이터베이스 서비스다. NoSQL은 관계형 테이블 구조에만 의존하지 않고 다양한 데이터 모델을 사용하는 데이터베이스를 통칭한다.

DynamoDB에서는 항목(Item)을 저장하고 기본 키(Primary Key)를 통해 데이터를 식별할 수 있다. 맥도날드에서는 주문 번호와 처리 상태를 기록하는 장부라고 생각하면 편하다.

주문번호	주문 상품	처리 상태
#101	빅맥	COMPLETED
#102	상하이버거	PENDING
#103	빅맥 세트	PENDING

이렇게 세 서비스의 역할을 나누면 다음과 같다.

API Gateway는 요청의 입구, Lambda는 실행 주체, DynamoDB는 데이터 저장소다.

3. 주문이 1,000개 몰리면 Lambda는 어떻게 할까?

이제 주문을 받는 구조는 만들었다. 그런데 점심시간이 되자 손님 1,000명이 동시에 몰렸다고 생각해 보자.

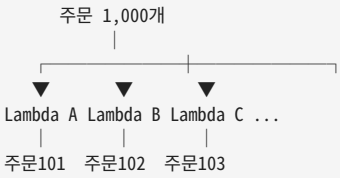
이 주문을 Lambda 하나가 전부 순서대로 처리해야 할까? 그렇지는 않다.

Lambda는 필요한 경우 여러 실행 환경을 만들어 요청을 동시에 처리할 수 있다. 이때 등장하는 개념이 **동시성(Concurrency)** 이다.

Concurrency - 동시성

동시성이란 여러 작업이 같은 시간 구간에 진행되는 성질을 말한다. AWS Lambda에서 동시 실행 수는 특정 시점에 실행 중인 함수 호출의 수를 의미한다.

맥도날드 직원이 한 명이라면 주문을 하나씩 처리해야겠지만 여러 직원이 있다면 각자 다른 주문을 동시에 처리할 수 있다.



물론 실제 실행 환경은 동시성 한도와 확장 속도의 영향을 받는다. 주문이 1,000개 들어왔다고 무조건 1,000개의 환경이 즉시 만들어지는 것은 아니다.

그런데 여기서 내가 처음 헷갈렸던 개념이 있다.

> "여러 Lambda가 동시에 실행되는 거면 이게 비동기 처리 아닌가?"

결론부터 말하면 **동시성과 비동기는 다른 개념**이다.

4. Synchronous와 Asynchronous

처음에는 동기 처리와 비동기 처리를 그저 융통성이 없고 유연한 방식 정도로 가볍게 생각했다.

그런데 정확히는 **작업 결과를 기다리는 방식**에 차이가 있었다.

Synchronous Processing - 동기 처리

동기 처리는 작업을 요청한 쪽이 해당 작업의 결과를 기다린 뒤 다음 처리를 이어가는 방식이다.

맥도날드 키오스크에서 빅맥을 주문했다고 생각해 보자. 주문 버튼을 누르자 화면에 처리 중이라는 문구가 나타난다. 그런데 햄버거가 완성될 때까지 키오스크에서 주문을 받았는지 말았는지 배고픈데 짜증나게 별 응답을 안돌려준다.

이까지가 동기 처리의 예시.

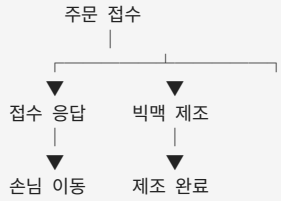
```
손님 → 주문 접수
        ↓
      빅맥 제조
       (10초)
        ↓
      제조 완료
        ↓
      최종 응답
```

Asynchronous Processing - 비동기 처리

비동기 처리는 요청한 작업의 완료를 기다리지 않고 다른 처리를 진행할 수 있도록 하는 방식이다. 이번에는 손님이 주문을 하고 키오스크에서 바로 주문 번호표를 받았다.

"102번 주문 접수됐습니다. 완성되면 알려드릴게요."

손님은 키오스크를 떠나 다른 일을 할 수 있다. 뭐 진짜 너무 배가 고프면 그냥 나가서 젤리에 맥주라도 하나 사먹고 올 수 있는거다. 주방에서는 그동안 빅맥을 만들고 있다.



여튼 중요한 건 비동기 처리가 빅맥을 더 빨리 만들어 주는 것은 아니라는 점이다. 10초 걸리는 햄버거는 여전히 10초가 걸릴 수 있다.

달라진 것은 **사용자가 첫 응답을 받는 시점**이다.

동시성과 비동기를 다시 비교

개념	핵심 질문	맥도날드 비유
동시성	여러 작업을 함께 진행할 수 있나?	직원 여러 명이 동시에 조리
동기 처리	결과가 나올 때까지 기다리나?	제조 완료 후 응답
비동기 처리	완료를 기다리지 않고 진행할 수 있나?	접수 후 별도로 제조

그러니까 직원 100명이 주문을 동시에 받더라도 손님 각각이 자기 햄버거의 완성 결과를 기다린다면 동기 처리일 수 있다. 반대로 직원이 한 명이어도 주문 접수와 제조 작업을 분리하면 비동기 처리 구조를 만들 수 있다.

동시성은 작업을 얼마나 함께 처리하는가의 문제이고 동기·비동기는 작업 결과를 어떻게 기다리는가의 문제다.

HTTP 201과 202

이 두 처리 방식을 HTTP API로 구현할 때는 응답 코드도 달라질 수 있다.

- **201 Created:** 요청에 따라 새로운 리소스가 생성되었음을 의미한다.
- **202 Accepted:** 요청이 처리를 위해 접수되었지만, 실제 처리는 아직 완료되지 않았을 수 있음을 의미한다.

여기서 201이 모든 후속 작업 완료를 뜻하는 것은 아니다. 이번 맥도날드 예시에서는 제조 완료를 나타내는 리소스를 생성한 후 201을 반환한다고 가정하자.

반면 비동기 방식에서는 주문 접수 직후 202를 반환할 수 있다.

결국 **주문을 접수했다는 것과 햄버거가 완성됐다는 것은 서로 다른 사건**이라는 점이 중요하다.

5. 주문은 받아뒀는데 주방이 감당하지 못하면?

자 이제 손님에게 번호표를 주고 먼저 보내는 데 성공했다. 그런데 또 다른 문제가 생길 수 있다.

주문이 100개 들어왔는데 주방에서는 한 번에 10개밖에 만들 수 없다면? 그렇다고 나머지 90개 주문을 없던 일로 할 수는 없으니까 어디엔가 보관해야 할 것이다.

여기서 **Event-Driven Architecture(이벤트 기반 아키텍처)** 가 등장한다.

이벤트 기반 아키텍처란?

이벤트 기반 아키텍처는 시스템에서 발생한 Event를 발행하고 다른 구성 요소가 그 이벤트를 전달받아 후속 작업을 수행하도록 구성하는 설계 방식이다.

예를 들어 주문 접수 이벤트가 발생하면 이를 전달받은 주방 시스템이 제조 작업을 시작하도록 만들 수 있다.

AWS에서는 EventBridge와 SQS 등을 이용해 이런 구조를 구현할 수 있다.

Amazon EventBridge

EventBridge는 이벤트를 수신하고 규칙에 따라 적절한 대상으로 전달하는 서버리스 이벤트 버스 서비스다.

맥도날드로 비유하면 새로운 주문이 들어왔다는 사실을 주방 시스템에 전달하는 역할이다.

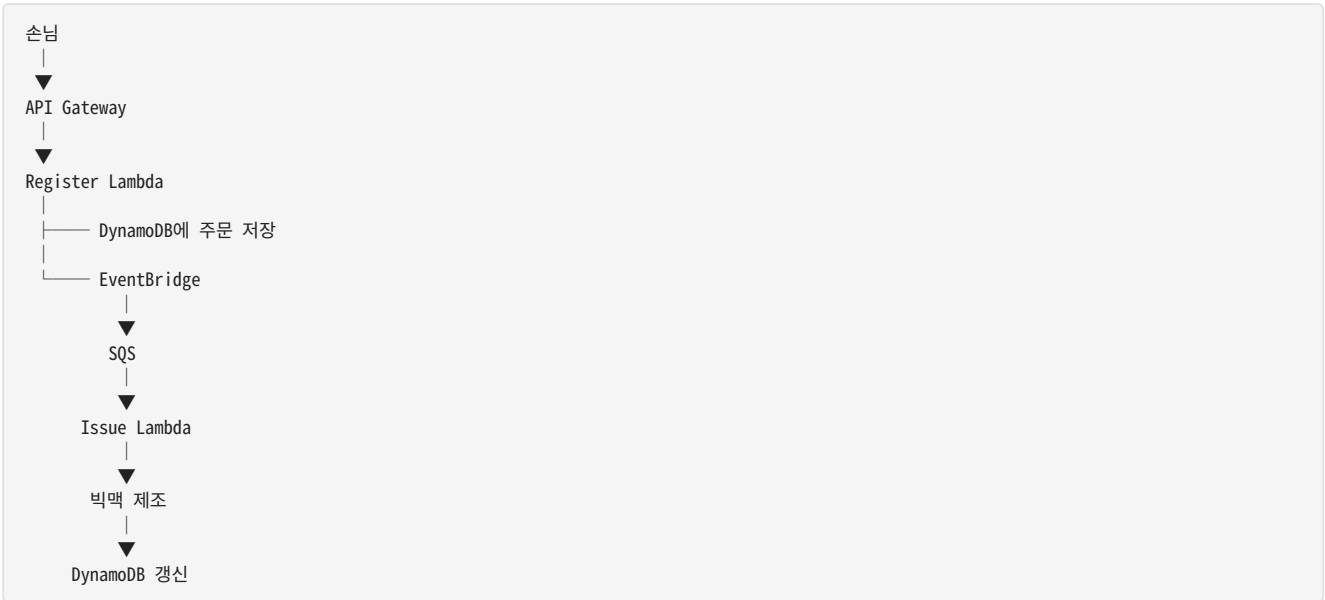
Amazon SQS

SQS(Simple Queue Service) 는 시스템 구성 요소 사이에서 메시지를 저장하고 전달하는 관리형 메시지 큐 서비스다. Queue는 먼저 들어온 데이터를 먼저 처리하는 FIFO 개념을 갖고 있는 점 기억.

다만 AWS의 SQS Standard Queue는 엄격한 선입선출 순서를 보장하지 않는다. 순서 보장이 필요하다면 FIFO Queue를 고려해야 한다고 한다.

맥도날드에서는 그냥 주문표를 쌓아두는 대기열이라고 생각하면 쉽다.

이벤트 전달을 담당하는 EventBridge와 메시지를 보관하는 SQS는 역할이 다르다.



실제로 SQS와 Lambda를 연동하면 Lambda의 이벤트 소스 매핑이 큐에서 메시지를 읽고 함수를 호출하는 방식으로 동작한다.

이렇게 구성하면 주문 접수 시스템과 제조 시스템을 분리할 수 있다. 이걸 **Decoupling(결합도 감소)** 이라고 한다.

한쪽의 처리 속도가 느리더라도 다른 쪽이 반드시 같은 속도로 동작할 필요가 없도록 만드는 것이다.

참고로 SNS(Simple Notification Service)는 하나의 메시지를 여러 구독자에게 전달하는 Pub/Sub 기반 서비스다. 주문 완료 사실을 여러 시스템에 알리는 용도 등으로 활용할 수 있다.

6. 주문표가 두 번 들어온다면 - 멱등성

이제 주문표를 SQS에 보관할 수 있게 됐다. 그런데 주문번호 102번의 빅맥을 이미 만들었는데 똑같은 주문표가 다시 들어왔다면?

주방 직원 입장에서는 황당할 것이다. "아니 이거 아까 만들었는데 또 만들라고?" 그런데 이런 상황은 실제 시스템에서도 충분히 발생할 수 있다.

SQS Standard Queue는 **At-Least-Once Delivery(최소 한 번 전달)** 방식을 사용하기 때문에 메시지가 중복 전달될 가능성이 있다.

따라서 메시지를 여러 번 받아도 결과가 중복 반영되지 않도록 설계해야 한다. 여기서 필요한 개념이 **Idempotency(멱등성)** 이다.

멱등성이란?

멱등성은 동일한 연산을 여러 번 수행해도 한 번 수행했을 때와 같은 최종 결과가 유지되는 성질이다. 맥도날드에선 같은 주문번호에 대한 제조 완료 기록이 여러 번 반영되지 않도록 하는 것이다.

```
주문 #102
    |
    ▼
제조 완료
    |
    ▼
COMPLETED / count = 1

같은 주문 #102 재전달
    |
    ▼
이미 처리된 주문 확인
    |
    ▼
중복 완료 기록 방지
    |
    ▼
COMPLETED / count = 1
```

그럼 DynamoDB에서 주문번호와 상태를 조회해서 이미 `COMPLETED`라면 무시하면 되는 것 아닐까?

나도 처음에는 그렇게 생각했는데 사실은 문제가 있었다. **만약 두 Lambda가 동시에 같은 주문을 처리한다면?**

7. 직원 두 명이 같은 빅맥을 만들기 시작했다.

주문번호 104번이 아직 `PENDING` 상태라고 생각해 보자. Lambda A와 Lambda B가 거의 동시에 이 주문을 조회했다.

A: "아직 안 만들었네? 내가 만들게."
B: "어? 나도 아직 안 만든 걸로 나오는데?"

결국 둘 다 같은 빅맥을 만들기 시작하는 것이다. 이런 상황이 **Race Condition** 이다.

Race Condition이란?

여러 실행 주체가 공유 데이터에 동시에 접근할 때 실행 순서나 타이밍에 따라 결과가 달라지는 문제를 말한다.

그렇다면 누군가 순서를 통제해 주면 되지 않을까? 운영체제에서 배웠던 Lock이나 Mutex처럼 말이다. 실제로 락을 사용하는 것도 동시성 문제를 해결하는 방법 중 하나다.

다만 Lambda는 서로 다른 실행 환경에서 동작할 수 있다. 따라서 특정 Lambda의 메모리에 만들어 놓은 일반적인 Lock만으로는 다른

Lambda 실행 환경까지 제어할 수 없다.

그래서 DynamoDB 자체가 원자적으로 상태 변경을 처리하도록 설계할 수 있다.

Atomic Operation

원자적 연산은 하나의 작업이 더 작은 중간 단계로 나뉘어 다른 연산과 뒤섞이지 않도록 처리되는 성질을 말한다.

예를 들어 주문 개수가 100개인데 Lambda A와 B가 동시에 값을 조회해 각각 101을 저장한다면 주문이 두 건 들어왔는데도 최종 값은 101이 되는 문제가 발생할 수 있다. 이를 Lost Update라고 한다.

DynamoDB에서는 Atomic Counter를 사용해 증가 연산 자체를 원자적으로 수행할 수 있다. 하지만 같은 주문을 여러 직원이 처리하지 못하게 하려면 조금 다른 기능이 필요하다.

바로 **Conditional Write(조건부 쓰기)** 다.

Conditional Write란?

조건부 쓰기는 지정한 조건이 참일 때만 데이터 변경을 허용하는 기능이다. 예를 들어 다음 규칙을 적용할 수 있다.

> "현재 주문 상태가 PENDING일 때만 COOKING으로 변경해."

여기서 `COOKING`은 내가 맥도날드 주문 시스템에 추가해 본 상태다. 실제 서비스에서는 `PROCESSING`이나 `IN_PROGRESS` 같은 이름을 사용한다.

```
PENDING
   |
   ▼
COOKING
   |
   ▼
COMPLETED
```

Lambda A와 B가 동시에 주문을 처리하려 해도 조건부 쓰기를 통해 하나의 Lambda만 `PENDING → COOKING` 변경에 성공하도록 만들 수 있다.

Lambda	시도	결과
A	PENDING → COOKING	성공
B	PENDING → COOKING	조건 불일치로 실패

이렇게 하면 적어도 같은 주문을 동시에 선점하는 문제를 줄일 수 있다.

원자적 연산은 동시 변경 과정의 안전성을 확보하고, 조건부 쓰기는 특정 조건에서만 변경하도록 제한하는 역할을 한다.

8. 그런데 빅맥 만들다가 Lambda가 죽으면..

이제 주문을 중복으로 선점하는 문제는 해결했다.

그런데 Lambda A가 104번 주문을 `COOKING`으로 바꾼 뒤 햄버거를 만들다가 갑자기 종료됐다고 생각해 보자. Lambda에는 실행 시간 제한인 **Timeout**이 있다.

Timeout은 함수 실행에 허용된 최대 시간을 의미하며 설정한 시간을 초과하면 실행이 중단된다. 그런데 DB에는 여전히 이렇게 남아 있다.

```
주문번호: #104
상태: COOKING
```

다른 Lambda가 주문을 다시 처리하려고 해도 이미 `COOKING`이라면 작업을 시작하지 못할 것이다. 그럼 이미 조리 중인 상태를 새 Lambda가 이어받으면 되지 않을까?

처음에는 헷갈려서 Lambda의 Warm Start가 이런 역할을 할 수 있다고 생각했다. 하지만 Warm Start와 장애 복구는 서로 다른 개념이다.

Cold Start와 Warm Start

Cold Start는 Lambda가 새로운 실행 환경을 초기화한 뒤 함수를 실행하는 상황을 말한다.

Warm Start는 기존 실행 환경을 재사용해 새로운 함수 호출을 처리하는 상황을 말한다.

구분	의미
Cold Start	새로운 실행 환경 초기화
Warm Start	기존 실행 환경 재사용
Retry	실패한 작업을 다시 시도

Warm Start는 중단된 함수를 이전 코드 위치부터 이어 실행하는 기능이 아니다. Lambda가 Timeout으로 실패했다면 해당 작업은 별도로 재시도하거나 복구해야 한다.

Stateless(무상태성)

여기서 또 중요한 개념이 **Stateless(무상태성)** 다.

Stateless란 각 요청을 처리할 때 이전 실행 환경에 저장된 상태에 의존하지 않도록 설계하는 성질이다. 예를 들어 맥도날드 직원이 자신이 만든 빅맥 개수를 머릿속에만 기억한다고 생각해 보자. 직원이 퇴근하고 새로운 직원이 오면 이전 직원의 기억을 그대로 전달받을 수는 없다.

Lambda도 마찬가지다.

```
total_orders = 0

def lambda_handler(event, context):
    global total_orders
    total_orders += 1
    return total_orders
```

같은 실행 환경을 재사용하면 `total_orders` 값이 남아 있을 수도 있다. 하지만 새로운 실행 환경에서는 다시 0부터 시작한다.

따라서 이전 호출의 메모리 상태가 유지된다는 가정으로 애플리케이션을 작성해서는 안 된다. 주문번호와 처리 상태처럼 지속되어야 하는 데이터는 DynamoDB 등의 영속 저장소에 기록해야 한다.

그럼 COOKING 상태는 어떻게 복구하지?

이때 사용할 수 있는 설계 중 하나가 **Lease(처리 기한)** 다.

Lease는 특정 작업을 일정 기간 동안 처리할 권한을 부여하고 기한이 지나면 다시 처리할 수 있도록 하는 방식이다.

예를 들어 다음과 같이 기록할 수 있다.

```
주문번호: #104
상태: COOKING
처리 담당: Lambda A
처리 기한: 14:30:30
```

Lambda A가 작업을 끝내지 못한 채 처리 기한이 만료되면 다른 Lambda가 조건부 쓰기를 통해 작업을 재선점하도록 설계할 수 있다.

물론 시간이 지났다고 자동으로 안전하게 복구되는 것은 아니다.

메시지 재전달이나 별도 복구 절차가 필요하고 이전 작업이 정말 중단되었는지와 중복 처리 가능성까지 고려해야 한다.

9. 빅맥은 이미 만들었는데 DB 기록 전에 죽었다면?

이제 마지막으로 조금 더 골치 아픈 상황을 생각해 보자.

Lambda A가 빅맥을 잘 만들어뒀다. 근데 DynamoDB에 `COMPLETED` 상태를 기록하려는 순간 Timeout으로 종료됐다. DB에는 여전히 `COOKING`이라고 남아 있다.

처리 기한이 만료되어 Lambda B가 다시 주문을 맡는다면 어떻게 될까?

이미 만들어진 빅맥을 또 만들 수도 있다!

```
주문 #104
    |
    ▼
Lambda A: 제조 완료
    |
    ▼
DB 기록 직전 Timeout
    |
    ▼
DB에는 COOKING
    |
    ▼
Lambda B 재시도
    |
    ▼
빅맥 중복 제조 위험
```

여기서 중요한 점이 있다.

DynamoDB에 `COMPLETED`를 한 번만 기록하도록 만드는 것과 실제 빅맥을 한 번만 제조하도록 보장하는 것은 별개의 문제다.

현실적인 시스템에서는 실제 외부 작업에도 주문번호 같은 고유 식별자를 전달하고, 이미 수행한 작업인지 확인할 수 있도록 설계해야 한다.

예를 들어 결제 시스템이라면 같은 결제 요청이 재전송되더라도 중복 결제가 발생하지 않도록 별도의 멱등성 처리가 필요하다.

멱등성은 DB 상태뿐 아니라 실제 작업에서 발생하는 부수 효과(Side Effect)까지 고려해야 한다.

Side Effect란 함수가 값을 반환하는 것 외에 DB를 변경하거나 외부 서비스에 요청하는 등 시스템 상태에 영향을 주는 동작이다.

10. 계속 실패하는 주문은 어떻게 처리할까?

이번에는 주문번호 105번을 처리하다가 주방 기계가 계속 고장 난다고 생각해 보자. 한 번 실패하고, 다시 시도해도 실패하고, 세 번째도 실패했다.

계속 같은 주문을 처리하려고 하면 문제가 해결되지 않을 수도 있다.

이때 필요한 것이 **Retry, Visibility Timeout, DLQ**다.

Retry(재시도)

Retry는 실패한 작업을 다시 수행하는 것이다. 일시적인 네트워크 오류나 서비스 장애처럼 시간이 지나면 해결될 수 있는 문제에 유용하다.

Visibility Timeout(가시성 제한 시간)

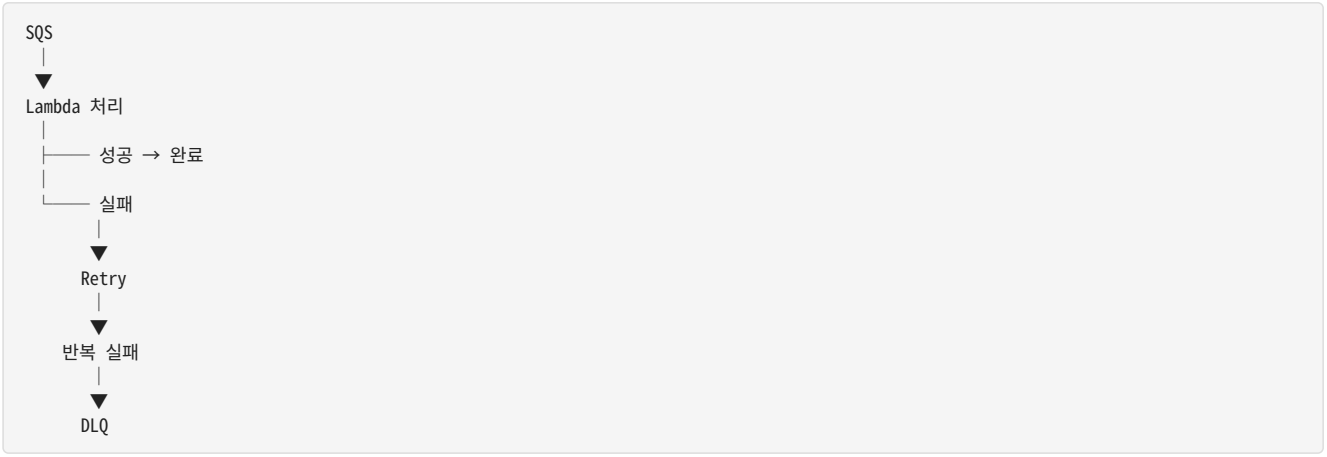
SQS에서 메시지를 소비자가 가져간 뒤 일정 시간 동안 다른 소비자에게 보이지 않도록 하는 설정이다.

맥도날드에서는 직원이 주문표를 가져가 조리하는 동안 다른 직원이 같은 주문표를 가져가지 않도록 잠시 숨기는 것과 비슷하다.

다만 Visibility Timeout만으로 메시지 중복 처리를 완전히 방지할 수는 없다.

DLQ(Dead Letter Queue)

DLQ는 반복적으로 처리에 실패한 메시지를 정상 큐에서 분리하여 보관하는 큐다.



계속 실패하면 그냥 버려야 하는 거 아닌가 싶은데 고객이 주문한 빅맥을 귀찮다고 없는 척 주문을 버려버리면 안된다..

DLQ는 메시지를 그냥 버리는 곳이 아니라 이런 문제 있는 작업을 격리해 원인을 분석하고 복구할 수 있도록 하는 공간이다.

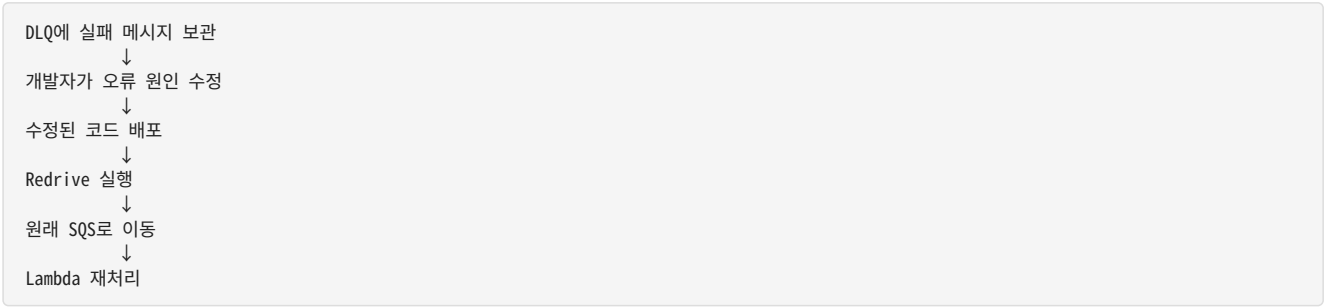
Redrive — 실패한 주문을 다시 보내기

그렇다면 오류를 수정하면 DLQ에 있던 메시지도 자동으로 처리될까? AWS가 인프라를 관리하니까 알아서 복구해주지 않을까?

하지만 DLQ에 들어간 메시지는 코드를 수정했다고 자동으로 원래 큐에 돌아오지 않는다.

이때 사용하는 기능이 **Redrive**

Redrive는 DLQ의 메시지를 원래 큐 또는 지정한 대상 큐로 다시 이동시켜 처리할 수 있도록 하는 기능이다.



여기서 조금 헷갈렸던 것이 Redrive의 주체였다. AWS가 운영을 대신한다면 개발자가 굳이 나서야 할까?

정리하면 **개발자나 운영자가 재처리 여부를 결정하고, SQS가 실제 메시지 이동 작업을 수행한다.**

물론 개발자가 사전에 자동화된 복구 절차를 구성할 수도 있다.

AWS는 개발자가 정해 놓은 규칙에 따라 동작을 수행하지만 어떤 오류가 해결되었는지 스스로 판단해 모든 작업을 복구해 주는 것은 아니다.

11. 데이터 정합성 — 빅맥은 나왔는데 주문 기록에는 없다고요?

여기까지 공부하고 보니 서버리스에서는 단순히 요청을 처리하는 것만큼 처리 결과를 정확히 기록하는 것도 중요하다는 생각이 들었다.

예를 들어 빅맥은 다 만들어졌는데 주문 장부엔 아직 대기 중이라고 기록되어 있다면?

주문번호	실제 상태	DB 상태
#101	제조 완료	COMPLETED
#102	제조 완료	PENDING
#103	제조 대기	PENDING

102번 주문에 문제가 발생했다.

이 상태에서 DB 기록만 믿고 다시 제조하면 빅맥이 두 개 만들어질 수도 있다.

여기서 등장하는 개념이 **Data Consistency(데이터 정합성·일관성)** 다.

데이터 정합성이란 데이터가 정해진 규칙과 제약조건에 맞게 서로 모순되지 않는 상태를 유지하는 것을 의미한다.

맥도날드 비유에서는 주문 장부에 기록된 상태가 실제 주문 처리 결과와 일치하도록 유지하는 문제로 이해할 수 있다.

다만 실제 조리 완료 여부처럼 DB 외부에서 발생한 결과까지 일치시키는 것은 단순한 DB 일관성보다 넓은 범위의 애플리케이션 정합성 문제긴 하다.

따라서 서버리스에서 데이터 정합성을 유지하려면 다음과 같은 점을 고려해야 한다.

- 동일 주문의 중복 반영 방지
- 여러 Lambda의 동시 수정 충돌 방지
- 작업 실패 시 상태 기록 관리
- 실제 외부 작업과 DB 기록의 불일치 복구

앞서 배운 멱등성, 조건부 쓰기, 상태 관리가 전부 여기와 연결되는 것이다.

12. IAM — 알바 첫날인데 금고 비밀번호까지 알려준다고?

이번에는 맥도날드에 신입 아르바이트생이 들어왔다고 생각해 보자. 이 직원의 업무는 주문을 확인하고 빅맥을 만드는 것이다.

그런데 매니저가 주문 장부 조회 권한뿐 아니라 매출 내역 삭제, 직원 급여 조회, 금고 개방 권한까지 전부 준다면?

아니 빅맥 하나 만들라고 뽑았는데 권력을 너무 잡는데?

AWS에도 비슷한 개념이 있다.

IAM(Identity and Access Management)

IAM은 AWS 리소스에 접근하는 주체를 식별하고, 어떤 작업을 수행할 수 있는지 권한을 관리하는 서비스다.

Lambda가 DynamoDB에 접근하려면 적절한 권한을 부여받아야 한다. 이때 Lambda가 사용하는 권한을 정의하는 IAM Role을 **Execution Role(실행 역할)** 이라고 한다.

예를 들어 주문 처리 Lambda에 다음과 같은 권한을 부여할 수 있다.

작업	IAM 권한 예시
주문 조회	`dynamodb:GetItem`
주문 저장	`dynamodb:PutItem`
주문 상태 수정	`dynamodb:UpdateItem`

그런데 단순히 주문을 처리하는 함수에 전체 DynamoDB 테이블을 삭제할 권한까지 부여할 필요는 없다.

이러한 보안 설계 원칙을 **Principle of Least Privilege(최소 권한 원칙)** 이라고 한다.

업무 수행에 필요한 최소한의 권한만 부여하는 것이다.

IAM 문제와 데이터 정합성은 연결될 수도 있다.

주방 직원이 빅맥을 완성하고 주문 장부에 완료 상태를 기록하려고 했다. 그런데 권한이 없어 `AccessDenied` 오류가 발생했다.

직접적인 원인은 IAM 권한 문제다.

하지만 실제 빅맥은 만들어졌는데 DB에는 여전히 `PENDING` 이 남았다면 데이터 정합성 문제로도 이어질 수 있다.

정리하면 다음과 같다.

- **IAM:** 이 작업을 수행할 권한이 있는가?
- **Data Consistency:** 작업 결과와 데이터가 일관되게 유지되는가?

서로 다른 개념이지만 실제 시스템에서는 하나의 문제가 다른 문제로 이어질 수 있는 것이다.

13. 그렇다면 서버리스에서 개발자의 역할은 무엇일까?

여기까지 공부하다 보니 처음의 질문으로 돌아오게 됐다.

> "AWS가 서버 운영을 대신해 준다면 개발자는 뭘 해야 하지?"

처음에는 그냥 코드 개선이나 기능 개발에 더 집중하면 되는 것 아닐까 싶었다.

틀린 말은 아니지만 이제는 조금 더 구체적으로 이해할 수 있을 것 같다.

AWS가 Lambda의 실행 환경을 관리하고 확장을 지원하더라도 애플리케이션이 올바르게 동작하는지는 별개의 문제다.

맥도날드에서 주방 시설과 직원 배치를 자동으로 관리한다고 해도 주문 누락, 중복 제조, 조리 실패, 직원 권한 문제까지 알아서 해결되는 것은 아닌 것처럼 말이다.

개발자가 고려할 것	관련 개념
주문을 동시에 처리하는 방법	Concurrency
접수와 제조를 분리하는 방법	Asynchronous Processing
주문을 안전하게 보관하는 방법	SQS
같은 주문의 중복 처리 방지	Idempotency
동시 수정 충돌 방지	Atomic Operation, Conditional Write
중단된 작업 복구	Retry, Lease
반복 실패 메시지 관리	DLQ, Redrive
처리 결과의 일관성 유지	Data Consistency
리소스 접근 권한 제한	IAM

물론 EC2를 사용한다고 이런 문제를 신경 쓰지 않아도 되는 것은 아니다.

다만 서버리스는 서버 인프라의 관리 부담을 AWS에 맡기는 만큼, 개발자가 애플리케이션의 로직과 신뢰성 설계에 더 집중할 수 있도록 해준다.

결국 서버리스에서 줄어드는 것은 모든 운영 책임이 아니라 **서버 인프라를 직접 관리해야 하는 부담**이다.

마무리

처음에는 Serverless를 그냥 평소에는 쉬다가 주문이 들어오면 그때그때 호출되는 방식 정도로 이해했고, Lambda가 자동으로 확장된다길래 비동기 처리랑 개념을 헷갈리기도 했다.

근데 막상 맥도날드 주문 시스템을 하나씩 구성해 본다고 생각하니까 생각보다 신경 쓸 게 많았다.

손님이 한꺼번에 몰리거나 주문은 받았는데 주방이 감당하지 못하면? 햄버거 이미 만들었는데 같은 주문표로 들어오면? 만들다가 Lambda가 죽어버리면..?

지피티쌤은 정말 명쌤이다. 다양한 상황을 구성해준 덕에 원래도 가장 궁금했던 개념인 서버리스에 대해서 좀 제대로 알게 된 것 같기도 하다. 배고파서 그런지 햄버거 이야기 하면서 velog 쓰니까 더 이해가 잘 되고 흡수도 잘된 느낌이다.

결국 서버리스는 서버를 누가 어떻게 관리하느냐의 문제이고, 동기와 비동기는 작업의 결과를 어떻게 기다리느냐의 문제였다. 또 AWS가 실행 환경을 관리해 준다고 해도 중복 처리나 데이터 정합성, 장애 복구 같은 부분은 여전히 개발자가 고민하고 설계해야 한다는 것도 알게 됐다.

물론 EC2를 사용한다고 이런 문제들이 없어지는 건 아니지만, 서버리스에서는 인프라 관리에 들이는 부담을 줄이고 애플리케이션 자체에 더 집중할 수 있다는 게 중요한 장점인 것 같다.

여튼 이번에도 맥도날드 덕분에 대충 개념의 틀은 잡힌 것 같다.